



Введение в Python

МФТИ 27/08/25 - 30/08/25

Буйлова Марианна - @Scarry_Mary

Переменные

Это ссылки на места в оперативной памяти, где лежит значение, которое может меняться — например, когда вы пишете после переменной знак равно (т.е. оператор присваивания) и какое-то новое значение.

```
In [ ]: a = 5
        b = 4

        print(a + b)

        # или: - так можно писать комментарии
        . . .
```

```
так тоже можно писать комментарии, чтобы ваш код был понятнее для других
'''
```

```
c = a + b
print(c)
```

```
9
9
```

Как ещё можно работать с операторами присваивания?

- Параллельное присваивание

```
In [ ]: a, b = 5, 3
```

```
print(a)
print(b)
```

```
5
3
```

- Обмен значений переменных

```
In [ ]: a, b = b, a
```

```
print(a)
print(b)
```

```
3
5
```

Типы данных

Рассмотрим какие типы данных есть в Python и какие операции для них доступны.

1. Целочисленный = `integer` = `int`

```
In [ ]: a = 7
        b = 2
```

Доступные операции:

- `+`: `a + b`
- `-`: `a - b`
- `:` `a b`
- `/`: `a / b`
- `//`: `a / b` - взятие целой части от деления

- %: a % b - взятие остатка от деления
- a**n - возведение в степень n

```
In [ ]: print ('a / b: ', a / b)
        print ('a // b: ', a // b)
        print ('a % b: ', a % b)
```

```
a / b:  3.5
a // b:  3
a % b:  1
```

1. Вещественные числа = float

- +: a + b
- -: a - b
- : a b
- /: a / b
- **n

```
In [ ]: c = 3.5
        print (a + c)
```

```
10.5
```

Так же можем ввести +-бесконечность:

```
In [ ]: x = float("-inf")
        print(x)
```

```
-inf
```

1. Логический тип данных = boolean = bool

Есть 2 значения - True, False

Поддерживает следующие операции: +, -, *

```
In [ ]: x = True
        y = False

        print('True + 3.5: ', x + c)
        print('False + 3.5: ', y + c)
```

```
True + 3.5:  4.5
False + 3.5:  3.5
```

Есть операторы для bool:

- not
- or
- and

```
In [ ]: print(not False)
```

True

```
In [ ]: print(True or False)
```

True

```
In [ ]: print(True and False)
```

False

<i>a</i>	<i>b</i>	<i>not a</i>	<i>a or b</i>	<i>a and b</i>
T	T	F	T	T
T	F	F	T	F
F	T	T	T	F
F	F	T	F	F

Вводим операторы сравнения:

- == - равны значения или нет
- '>' или '>=' ('>' стоит из-за особенностей Markdown(!), в питоне просто >)
- '<' или '<='
- '!=' - отсутствие равенства значению

```
In [ ]: a = 1
        b = 3

        print('a == 4: ', a == 4)
        print('b > a: ', b > a)
        print('a >= b: ', a >= b)
        print('a != 5: ', a != 5)
```

```
a == 4: False
b > a: True
a >= b: False
a != 5: True
```

1. Строки = string = str

- Можно складывать, но не с числами
- Можно умножать на число

```
In [ ]: c = 'Hello'
        d = 'World'

        print('c * 5: ', c * 5)
        print('c + d: ', c + d)
```

```
c * 5:  HelloHelloHelloHelloHello
c + d:  HelloWorld
```

Можем превращать числа и bool-переменные в строки! Перед этим введем функцию `type()`, которая поможет нам определять тип данных, с которыми имеем дело.

```
In [ ]: a = 5
        b = str(5)
        print(type(a))
        print(type(b))
```

```
<class 'int'>
<class 'str'>
```

```
In [ ]: c = True
        d = str(c)
        print(type(c))
        print(type(d))
```

```
<class 'bool'>
<class 'str'>
```

Как считать данные из консоли?

До этого мы ручками определяли переменные, а что, если их должен ввести другой пользователь? В этом нам поможет **`input()`**!

```
In [ ]: a = input()
```

```
a
```

Можем узнать тип переменной, используя функцию **`type()`**, в которую помещаем нашу переменную

```
In [ ]: type(a)
```

Out[]: str

input() всегда считывает строку, но мы можем переопределить тип данных с помощью функций:

- int()
- float()
- bool()

```
In [ ]: b = input()
```

5

```
In [ ]: print(type(int(b)))
```

<class 'int'>

```
In [ ]: print(type(float(b)))
```

<class 'float'>

```
In [ ]: print(type(bool(b)))
```

<class 'bool'>

Пример №1

Внутри скобочек у input ты пишешь пояснение для юзера, что конкретно его просят ввести. Но вот беда, по умолчанию все, что вводится через input, является не числом, а строкой, поэтому, прежде чем складывать количество литров кофе, нужно сначала преобразовать введенные строки в числа с помощью функции int().

```
In [ ]: coffee1 = input('Введи, сколько у тебя литров кофе: ')
coffee2 = input('Сколько кофе принес друг: ')
coffee3 = int(coffee1) + int(coffee2)
answer = 'На двоих у вас: ' + str(coffee3) + ' литров кофе'
print(answer)
```

Введи, сколько у тебя литров кофе: 2
Сколько кофе принес друг: 5
На двоих у вас: 7 литров кофе

Пример №2

Вот еще пример, рассчитывающий, сколько тебе осталось пить кофе, исходя из средней продолжительности жизни в России:

```
In [ ]: age = input('Введи, сколько тебе лет: ')
max_age = 73 - int(age)
print('Осталось примерно: ' + str(max_age) + " лет")
```

Введи, сколько тебе лет: 22
Осталось примерно: 51 лет

Оператор присваивания

Вернёмся к оператору присваивания. Что происходит при инициализации переменной?

Например, создаётся целочисленная переменная **b = 5**,

(1) Создаётся "ячейка" и 5 кладётся в эту ячейку.

(2) Данный объект имеет некоторый **идентификатор**, **значение** 5 и его **тип** - целое число (integer, int).

(3) Посредством оператора присваивания ("=") создаётся ссылка между переменной b и целочисленным объектом 5.

Что же за идентификатор? - рассмотрим **функцию id()**

```
In [ ]: b = 5
id(b)
```

Out[]: 11642216

```
In [ ]: b = 6
id(b)
```

Out[]: 11642248

Поменялось значение переменной - поменялся и id, чтобы такого не произошло, переменная должна быть **изменяемого** типа данных. Делаем вывод - int переменные относятся к **неизменяемому** типу данных. Рассмотрим следующую таблицу:

Неизменяемые типы данных	Изменяемые типы данных
int - целые числа	list(списки)/массивы
float - вещественные	
bool - True/False	
str - строки	

Пока мы познакомились с неизменяемыми типами данных, позже изучим

изменяемые, например, массивы.

Итак, операторы присваивания всех видов:

+=	a += 1 эквивалентно a = a + 1
-=	аналогично, a -= 1 <=> a = a - 1
*=	a = 2 <=> a = a * 2
/=	a /= 2 <=> a = a / 2 обычное деление
%=	a %= 2 <=> a = a % 2, взятие остатка от деления
//=	a //= 2 <=> a = a // 2, взятие целой части от деления
**=	a = 2 <=> a = a ** 2, возведение в степень

Пример:

```
In [ ]: b = 5
        b //= 2
        print(b)
```

2

```
In [ ]: b **= 2
        print(b)
```

4

Обмен переменных значениями

Рассмотрим 2 варианта, как ещё это можно сделать.

```
In [ ]: a = 2
        b = 5
        tmp = a
        a = b
        b = tmp
```

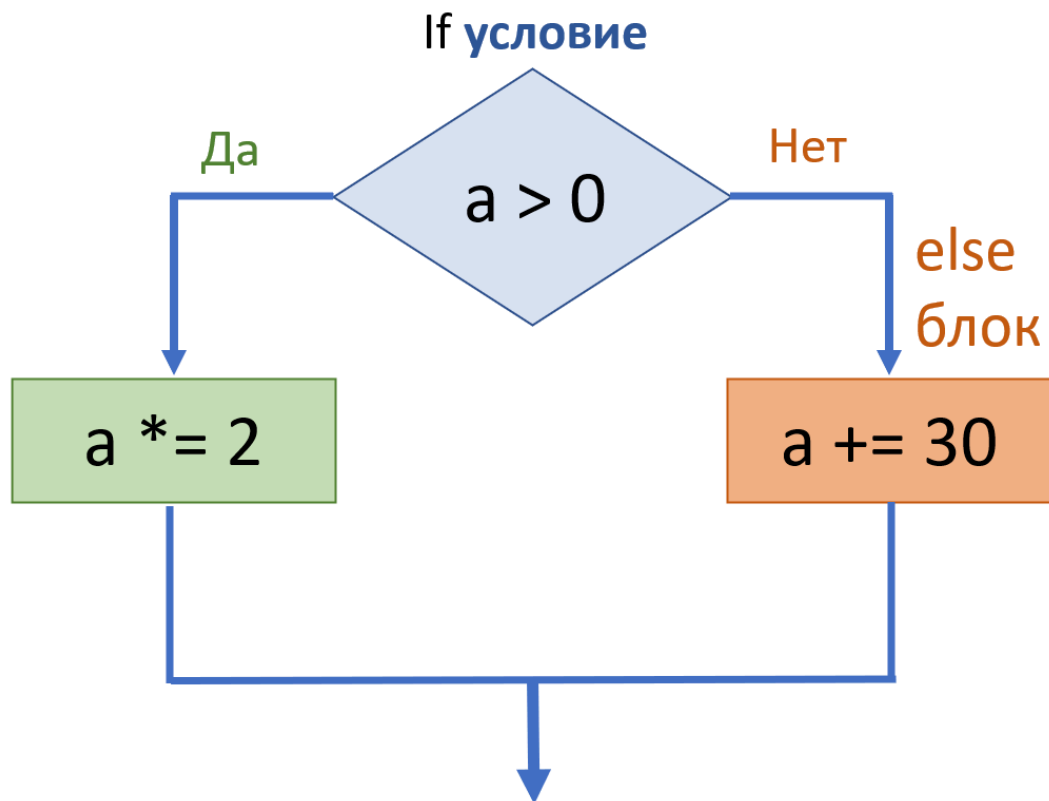
```
In [ ]: a = 2
        b = 5
        tmp1 = b
        tmp2 = a
        a = tmp
        b = tmp2
```

Ветвления/условия (if, elif, else)

В основе любой программы лежат условия. В зависимости от того, выполняются они или не выполняются, программа может пойти по одному или другому пути. Ветвления бывают простыми и вложенными, рассмотрим оба случая.

В строчке после **if** идет условие, которое мы проверяем. Если оно верно, выполняется первый блок кода (он отделен четырьмя пробелами вначале). Если неверно, то тот, что после **else**.

Пример простого ветвления:



```
In [2]: a = int(input())
```

```
if a > 10:
    a *= 2
else:
    a += 30

print(a)
```

10
40

Более сложный пример:

```
In [ ]: coffee = input('Введите Yes, если кофе есть, и No, если кофе нет: ')
if coffee == 'Yes':
    answer = 'Ты взломаешь Пентагон'
else:
    answer = 'Ты сломаешь свой мозг'
print(answer)
```

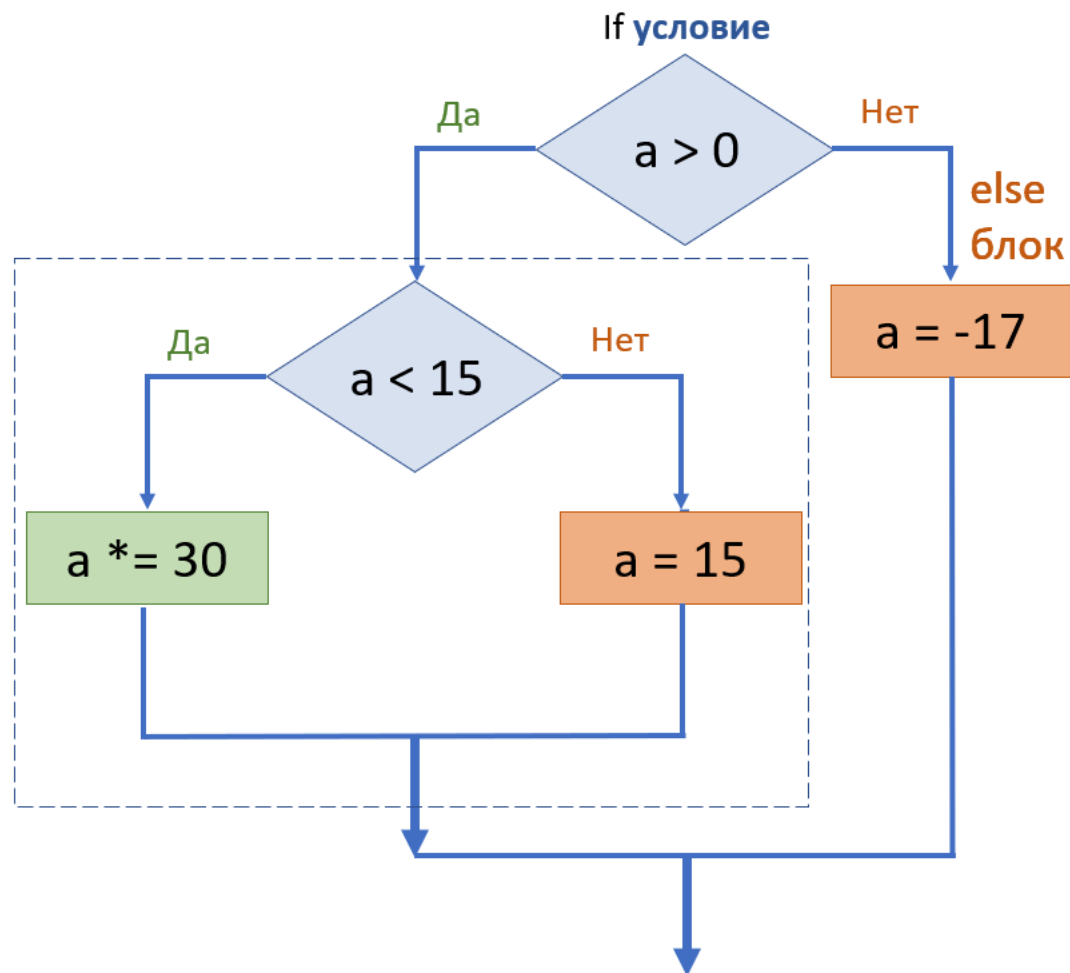
Введите Yes, если кофе есть, и No, если кофе нет: Yes
Ты взломаешь Пентагон

Еще один важный момент здесь — это знак равенства в условии. Он пишется как двойное «равно» (==) и этим отличается от присвоения — одинарного «равно».

```
In [ ]: coffee = input('Введите Yes, если coffee есть, и No, если coffee нет: ')
if coffee.lower() == 'yes':
    answer = 'Ты взломаешь Пентагон'
else:
    answer = 'Ты сломаешь свой мозг'
print(answer)
```

Функция **lower()**, прежде чем сравнивать условие, делает все буквы в строке маленькими, потому что глупый юзер может ввести слово YES с горящим Caps Lock, и это надо предусмотреть заранее.

Пример вложенного ветвления



```
In [ ]: a = int(input())
```

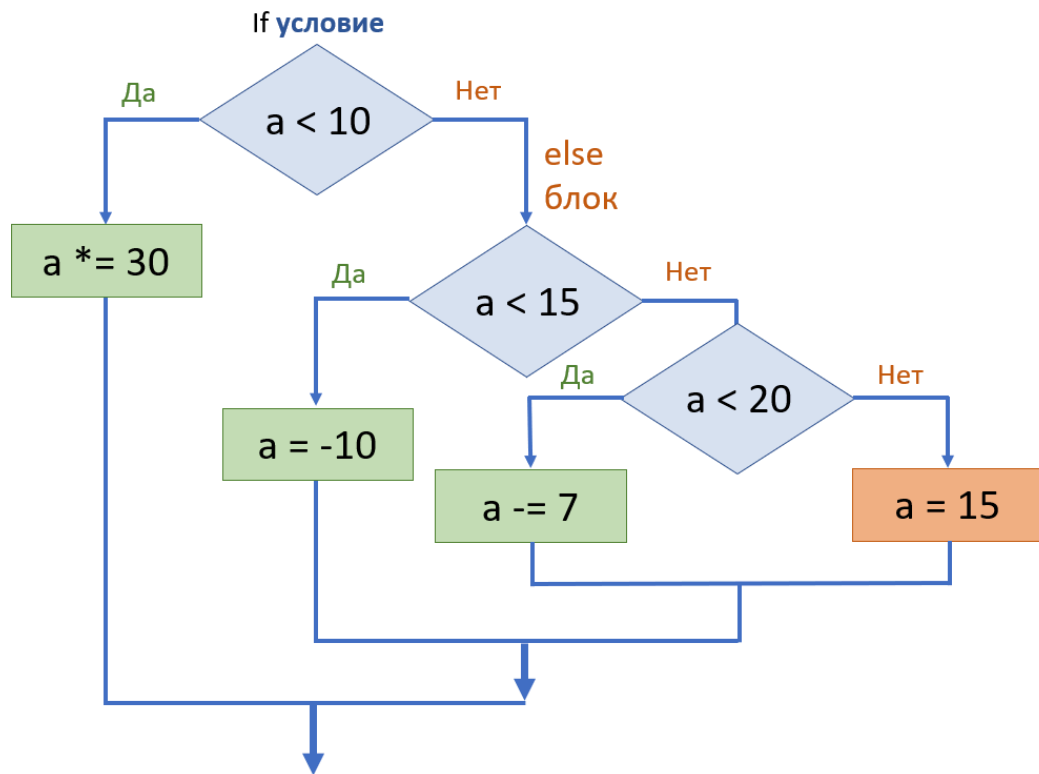
```
if a > 0:
    if a < 15:
        a *= 30
    else:
        a = 15
else:
    a = -17
```

```
print(a)
```

```
100
```

```
15
```

Каскадное ветвление



In [1]: `a = int(input())`

```

if a < 10:
    a *= 30
else:
    if a < 15:
        a = -10
    else:
        if a < 20:
            a -= 7
        else:
            a = 15

```

`print(a)`

45

15

Как сделать проще? **elif**

In [3]:

```

if a < 10:
    a *= 30
elif a < 15:
    a = -10
elif a < 20:
    a -= 7
else:

```

```
a = 15
```

Цикл while

Цикл **while** работает до тех пор, пока выполняется условие, указанное справа:

```
In [ ]: x = int(input())
```

```
while x > 5:  
    x -= 1  
    print(x)
```

```
10  
9  
8  
7  
6  
5
```

Пример 1: Подсчитать число элементов в последовательности

```
In [ ]: a = input()  
count = 0  
  
while a != 'end':  
    count += 1  
    a = input()  
  
print('Count: ', count)
```

```
3  
3  
end  
Count 2
```

Пример 2: Поиск максимума

```
In [ ]: a = input()  
m = 12345  
  
while a != 'end':  
    a = int(a)  
  
    if a > m:  
        m = a  
  
    a = input()
```

```
print('Maximum: ', m)
```

```
1233
1234567
end
Maximum: 1234567
```

Пример 2.2: Поиск максимума с использованием флага

```
In [ ]: f = False
m = float('-inf')
a = input()

while a != 'end':
    if not f or int(a) > m:
        m = int(a)
        f = True
    a = input()

print('Maximum: ', m)
```

```
2
3
0
-1
5
end
Maximum: 5
```

Цикл for

Перебор элементов последовательности

```
In [ ]: for i in 0,1,2,3,4:
        print(i)
```

```
0
1
2
3
4
```

Перебор элементов строки

```
In [ ]: for x in 'word':
        print(x)
```

```
w
o
r
d
```

Познакомимся с функцией `range()` - генератором арифметической прогрессии.



Итерация - один проход цикла/одно из повторений цикла.

Рассмотрим ещё 2 команды, с которыми необходимо ознакомиться при работе с циклами.

1) **break** - досрочно прерывает цикл

```
In [1]: for i in range(10):  
        if i > 5:  
            break  
        print(i)
```

```
0  
1  
2  
3  
4  
5
```

2) **continue** - переключается на следующую итерацию

```
In [2]: for i in range(10):  
        print(i)  
        if i > 5:  
            continue  
        print('Hello!')
```

```
0
Hello!
1
Hello!
2
Hello!
3
Hello!
4
Hello!
5
Hello!
6
7
8
9
```

3) **else**

```
In [3]: for e in range(5):
        print(e**e)
        else:
            print('end')
```

```
1
1
4
27
256
end
```

В контексте цикла while тоже можно использовать break & continue

```
In [5]: x = -5
        while x < 0:
            x += 1
            if x == -1:
                break
            print(x)
```

```
-4
-3
-2
```

```
In [6]: x = -5
        while x < 0:
            x += 1
            print(x)
            if x > -3:
                continue
            print('Hi')
```

-4
Hi
-3
Hi
-2
-1
0

Функции

имя функции аргументы



```
def sum(x, y):  
    z = x + y  
    return z
```

Как использовать?

(1)

```
In [7]: def sum(a, b):  
        c = a + b  
        return c
```

```
In [9]: x = int(input())  
        y = int(input())  
  
        z = sum(x, y)  
        print(z)
```

3
4
7

(2)

```
In [11... def sum(x, y):  
    print(x + y)
```

```
In [12... def sum(x, y):  
    print(x + y)  
  
x = int(input())  
y = int(input())  
  
print(sum(x, y))
```

```
1  
2  
3  
None
```

None - помечает отсутствие данных, в нашем случае так происходит, потому что функция sum ничего не возвращает, она только печатает результат сложения, но именно переменную не возвращает.

(3)

```
In [15... def abs1(x):  
    if x < 0:  
        return -x  
    return x  
  
def abs_sum(a, b):  
    return abs1(a) + abs1(b)  
  
x = int(input())  
y = int(input())  
  
print(abs_sum(x,y))
```

```
2  
-2  
4
```

(4)

```
In [17... def sum_sub(a, b):  
    return a + b, a - b  
  
x = int(input())  
y = int(input())  
  
a, b = sum_sub(x,y)  
print(a, b)
```

3
-1
2 4

(5) Значения по умолчанию

```
In [18... def sum(a, b=5):  
            return a + b  
  
print(sum(15))  
print(sum(12,3))
```

20
15

Во втором случае значение по умолчанию не пригодились.

Аргументы со значениями по умолчанию идут после всех остальных аргументов.

Как делать не надо!

```
In [ ]: def sum(a, b=5, c):  
         return a + b + c
```

Можно:

```
In [20... def sum(a, b, c=5):  
            return a + b + c
```

```
In [22... def sum(a=7, b=8, c=5):  
            return a + b + c  
  
print(sum())
```

20

(6)

```
In [24... def f():  
            return 'Hello'  
  
print(f())
```

Hello

(7)

```
In [25... def sum(a=7, b=8, c=5):  
            return a + b + c
```

```
print(sum(c=0))
print(sum(b=6, a=3))
```

15

14

(8) Пример: проверим, является ли число простым

```
In [27... def is_prime(x):
    for i in range(2, x):
        if x % i == 0:
            return False
    return True

x = int(input())
print(is_prime(x))
```

5

True

(9)*** Разложение числа на множители

```
In [29... def factorize(x):
    for i in range(1, round(x**0.5) + 1):
        if x % i == 0:
            print(i, x // i)

x = int(input())
factorize(x)
```

256

1 256

2 128

4 64

8 32

16 16

Массивы/списки

Как можно создать?

- Перечислением элементов: `a = [1, 2, 3, 4]`
- Заполнением нулями: `a = [0] * n`
- Функцией `list()`: `a = list()`

(1) Генерация массива из нулей

```
In [31... n = int(input())
a = [0] * n

for i in range(n):
```

```
a[i] = input()
print(a)
```

```
3
1
5
7
['1', '5', '7']
```

(2) Создание массива из строки

```
In [34... s = input().split()

for i in range(len(s)):
    s[i] = int(s[i])

print(s)
```

```
1 2 4
[1, 2, 4]
```

(3) Функция map

```
In [36... s = input().split()
s = list(map(int, s))

print(s)
```

```
1 2 3
[1, 2, 3]
```

(4) Добавление элементов

```
In [37... s.append(4)
print(s)
```

```
[1, 2, 3, 4]
```

(5) Удаление элемента (с конца)

```
In [38... s.pop()
print(s)
```

```
[1, 2, 3]
```

Если хотим извлечь конкретный элемент, то в () ставим его номер.

```
In [39... s.pop(1)
```

```
Out[39]: 2
```

(6) Добавление элемента на определённое место

a.insert(position, element)

```
In [41... a = [4, 6, 7]
a.insert(1, 5)

print(a)
```

[4, 5, 6, 7]

```
In [42... a.insert(3, 'g')

print(a)
```

[4, 5, 6, 'g', 7]

(7)*** Более сложный способ заполнить/сгенерировать массив => списковое включение

```
In [43... a = [i for i in range(10)]

print(a)
```

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```
In [45... a = [i**2 for i in range(10)]

print(a)
```

[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

```
In [46... a = [i for i in range(10) if i % 2 == 0]

print(a)
```

[0, 2, 4, 6, 8]

(8) Срезы

array[start : end : step]

По умолчанию — 0 ————— 1 —

Последний
элемент, не
включительно

```
In [1]: a = [4, 5, 7, 6, 3]
```

```
print(a[:3])  
print(a[::-1])
```

```
[4, 5, 7]
```

```
[3, 6, 7, 5, 4]
```

(9)** срезы для строк

```
In [4]: s = 'Hello, world!'
```

```
print(s[1::2])  
print(s[::-1])  
print(s[-4])
```

```
el, wrd
```

```
!dlrow ,olleH
```

```
r
```

Пример: поиск суммы чётных положительных элементов в массиве

```
In [47... n = int(input())  
a = [int(input()) for _ in range(n)]  
s = 0
```

```
for e in a:  
    if e > 0 and e % 2 == 0:  
        s += e  
print(s)
```

```
2
```

```
1
```

```
3
```

```
0
```

(8) Можно пройтись и по объектам массива, и по номерам

```
In [48... a = 'psppspspsp'
```

```
for i, e in enumerate(a):  
    print(i, e)
```

```
0 p
1 s
2 p
3 p
4 s
5 p
6 s
7 p
8 p
9 s
10 p
11 s
```

In [49...

```
a = 'psppspspsp'
b = '546474848484'

for x,y in zip(a,b):
    print(x, y)
```

```
p 5
s 4
p 6
p 4
s 7
p 4
s 8
p 4
p 8
s 4
p 8
s 4
```

Пасхалка №1! тест: <https://sky.pro/media/tests/test-na-znanie-python/?ysclid=mey4hwjp30801748245>

